

Highlights

Limits of Fibonacci Index-Gap Coding: Complete Stream Costs, Compression Bounds, and Structural Controls

Stéphane Gaël R. EKODECK, Hervé Talé KALACHI, Serge Alain EBÉLÉ, Norbert DJONG WANG, Chantal Marguerite MVEH-ABIA

- FISA 1.0 serializes every field required to reconstruct the input.
- An exact inequality identifies when its shared stream beats raw storage.
- The optimistic favorable fraction reaches 40.18% before framing costs.
- Complete-stream ratios range from 0.9215 to 0.9878 at the best tested sizes.
- Binary-position and zero-structure controls explain the measured savings.
- Established codecs win every paired comparison on Canterbury and Silesia.

Limits of Fibonacci Index-Gap Coding: Complete Stream Costs, Compression Bounds, and Structural Controls

Stéphane Gaël R. EKOdeck^{a,c,d,*}, Hervé Talé KALACHI^{a,b}, Serge Alain EBÉLÉ^{a,c,d}, Norbert DJONG WANG^a and Chantal Marguerite MVEH-ABIA^{a,b}

^aLIRIMA, TEAM GRIMCAPE, University of Yaounde I, Yaounde, Cameroon

^bNational Advanced School of Engineering (Polytechnic), University of Yaounde I, Yaounde, 8390, Cameroon

^cIRD, UMI, UMMISCO, IRD France Nord, Bondy, 93143, France

^dSorbonne Universités, Univ. Paris 06, UMI 209, UMMISCO, Paris, 75005, France

ARTICLE INFO

Keywords:

Lossless compression
Fibonacci coding
Zeckendorf representation
Integer codes
Transform coding
Reproducible evaluation

ABSTRACT

Fibonacci index gaps can look shorter than direct binary integers while still producing a larger decodable stream. We examine where that discrepancy comes from. The test format, Fibonacci Index-gap Shared Alphabet (FISA) 1.0, maps each block to gaps between its Zeckendorf indices, shares one alphabet across the file, and falls back to raw storage when needed. Its serialized size includes the alphabet, mode and length fields, block records, and padding. The gap map is bijective, and an exact inequality states when the shared stream is shorter than the raw alternative. Enumerating every integer through 10^7 gives an optimistic favorable fraction of 40.18%, but this comparison omits framing. On complete files, block size materially changes the outcome: Canterbury reaches its best tested FISA ratio at 32 bytes (0.9215), whereas Silesia reaches 0.9878 at 256 bytes. Only one Canterbury file and at most two Silesia files compress at any tested size. At 256 bytes, just 3.70% and 1.94% of blocks satisfy the sufficient bound. Simpler controls are smaller after their own block-size sweeps: zero trimming reaches 0.8942 on Canterbury and a zero-block bitmap reaches 0.9774 on Silesia. Ordinary binary set-bit gaps also dominate the optimistic integer comparison, and established codecs win all 11 Canterbury and all 12 Silesia file comparisons. These results delimit, rather than promote, the Fibonacci construction: the observed savings arise mainly from positional sparsity and zero structure, not from the number system alone.

1. Introduction

Consider a transform that turns a byte block into a shorter list of integers. That list is not yet a compressed file. The decoder still needs boundaries, lengths, selectors, and often a model. In our experiments, those bytes decide the result. We use “complete cost” for every byte inside the serialized stream boundary. The reusable decoder, its working memory, and execution time are measured separately.

We examine one concrete transform. Zeckendorf decomposition writes an integer as a unique sum of non-consecutive Fibonacci terms [1]. Replacing the selected indices by adjacent differences gives an invertible gap sequence. FISA, the Fibonacci Index-gap Shared Alphabet format, stores these gaps with one alphabet for the file. The format is byte-exact: it preserves leading zeros, records every length needed by the decoder, and rejects malformed input.

The first question was simple: does the short gap sequence survive serialization? We derived an exact stream-gain condition and a sufficient bound on the number of selected terms. We then enumerated integers through 10^7 and ran complete-corpus tests at five block sizes. The integer

count looked promising. The file results did not. Only a few files compressed, and the best block size changed with the corpus.

That mismatch led to a second question. Was Fibonacci responsible for the small gains? We repeated the optimistic comparison with Lucas indices and ordinary binary set-bit positions. We also built two direct zero-structure controls. Binary positions produced shorter gap lists for most tested integers, while zero trimming and a zero-block bitmap beat FISA on the favorable corpora. We use FISA as a documented test case for complete stream accounting, not as a replacement for established codecs.

The remainder of the paper is organized as follows. Section 2 discusses related coding approaches. Section 3 establishes the mathematical notation and worked examples. Section 4 defines the transform and stream. Section 5 gives correctness and complexity results. Section 6 describes the experimental protocol. Section 7 reports the compression evaluation. Section 8 discusses scope and limitations.

2. Related work

2.1. Entropy and general-purpose coding

Huffman coding remains a standard reference for symbol coding with integer-length codewords [2]; standard treatments place such codes within statistical, dictionary, and context modelling [3, 4]. Transform coding can preserve information while exposing a representation better matched to a subsequent model, as in block sorting [5, 6]. Zstandard

*Corresponding author

✉ stephane-gael.ekodeck@facsciences-uy1.cm (S.G.R. EKOdeck);
herve.tale@univ-yaounde1.cm (H.T. KALACHI);
djongwangnorbert@gmail.com (N.D. WANG); cmmveh@yahoo.fr (C.M. MVEH-ABIA)

ORCID(s): 0000-0002-8094-8832 (S.G.R. EKOdeck);
0000-0002-2590-4159 (H.T. KALACHI); 0000-0001-7036-7068 (S.A. EBÉLÉ)

illustrates the practical importance of jointly considering ratio, framing, and speed [7], while current work on Lempel–Ziv parsing continues to improve construction and representation methods [8]. These methods learn or estimate symbol regularities across a sequence, whereas the positional transform considered here first maps each byte block to a single large integer.

2.2. Fibonacci and integer representations

Fibonacci universal codes represent positive integers with a terminator and are useful where self-synchronization and direct integer decoding matter. The underlying unique non-consecutive representation is due to Zeckendorf [1], and robust complete Fibonacci code families were studied by Fraenkel and Klein [9]. Practical indexing studies compare their space and decoding behavior with other universal codes [10]. Recent work continues to compare schemes for natural-number sequences, including Fibonacci, Elias-family, and related representations [11, 12]. Generalized Fibonacci number systems have also been studied beyond one-dimensional integer coding [13]. Domain-specific lossless compressors provide a separate reference point: Gorilla exploits XOR structure in floating-point time series [14], while Chimp improves reference selection and significant-bit coding [15]; Sprintz provides a domain-specific reference point for predictive coding on resource-constrained IoT streams; it is used for positioning rather than as an experimental baseline [16].

The proposed representation is not a conventional Fibonacci universal code. It first obtains the indices of the Fibonacci terms in a block integer and then codes differences between adjacent indices. This places the method closer to gap coding used in compressed indexes, where the distribution of differences determines whether a universal or entropy code is effective.

2.3. Positioning relative to the evaluated state of the art

Table 1 distinguishes the coding role of each family. Fibonacci universal codes address positive integers, Huffman codes symbols under a probability model, and Lempel–Ziv-class codecs exploit repeated substrings. FISA 1.0 is evaluated as a blockwise front-end representation with a complete codebook and a raw fallback.

Positioning statement. The contribution is a reversible representation layer, not a replacement entropy coder or dictionary model. Its direct comparison targets are universal integer codes at the transformed-stream level and mature lossless codecs at the complete-file level.

3. Preliminaries

This section establishes the mathematical objects used throughout the paper: the Fibonacci sequence, Zeckendorf decomposition, and the gap reduction that forms the basis of the FISA transform.

Input: Non-negative integer d

Output: Exact Fibonacci values covering d

$V \leftarrow [0, 1, 1];$

while $V[-1] \leq d$ **do**

 append $V[-1] + V[-2]$ to V ;

end

return V ;

Algorithm 1: FibonacciTable(d)

3.1. Fibonacci sequence

The Fibonacci sequence is defined by

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad (n \geq 2). \quad (1)$$

Its closed form is

$$F_n = \frac{\phi^n - \psi^n}{\sqrt{5}}, \quad \phi = \frac{1 + \sqrt{5}}{2}, \quad \psi = \frac{1 - \sqrt{5}}{2}. \quad (2)$$

Equation (2) explains the growth $F_n = \Theta(\phi^n)$, but the codec does not evaluate it numerically: floating-point rounding cannot guarantee exact reconstruction for large indices. Required values are generated by exact integer addition, and fast doubling is used for random access. Algorithm 1 constructs the exact table required for greedy decomposition.

For example, $F_{10} = 55$ and $F_{20} = 6765$. The index grows linearly while the represented value grows exponentially. This motivates representing selected terms by indices rather than by their values.

3.2. Notation

Table 2 summarizes the symbols used in the transform and its analysis.

Table 3 gives the abbreviations retained in the remainder of the paper.

3.3. Integer decomposition via Fibonacci indices

3.3.1. Basic term decomposition

Greedy decomposition repeatedly selects the largest Fibonacci value not exceeding the current remainder. For $d = 18324$,

$$18324 = F_{22} + F_{15} + F_4 = 17711 + 610 + 3. \quad (3)$$

The direct binary representation requires 15 bits. Writing the selected Fibonacci values separately requires the optimistic sum $15 + 10 + 2 = 27$ bits, before delimiters, so this example expands.

3.3.2. Term indices

The same decomposition has index sequence $I = [22, 15, 4]$. The optimistic sum of index bit lengths is $5 + 4 + 3 = 12$ bits. This motivates index storage, but it is not yet a complete code because element boundaries remain unspecified.

Table 1

Methodological positioning relative to the evaluated state of the art.

Family	Coding unit	Structure exploited	Side information	Position in this study
Huffman coding [2]	Symbols	Symbol probabilities	Tree, table, or probability model	Reference entropy-coding layer; not replaced by the index-gap transform
Lempel–Ziv, LZMA, Brotli, and Zstandard [7, 8, 17, 18]	Byte strings	Repeated substrings and local matches	Dictionary references and framing	General-purpose reference class; zlib, LZMA, Brotli, bzip2, Zstandard, and PPMd are measured
Universal integer and Fibonacci codes [10, 11, 19, 20]	Positive integers or gaps	Integer magnitude and self-delimitation	Terminators or code boundaries	Direct conceptual baseline; measured payload ratios exceed one on both corpora
FISA 1.0 (framework instance)	Fixed byte blocks	Sparse Zeckendorf indices and repeated index gaps	Alphabet, lengths, identifiers, and mode	Exact conditional representation with raw fallback outside favorable blocks
Zero-structure baselines (this evaluation)	Fixed byte blocks	Empty blocks or boundary zero runs	Presence bitmap or per-block trim lengths	Simpler controls that delimit the gain attributable to Fibonacci indices

Note: The last column contains only conclusions supported by this study.

Table 2

Main notation.

Symbol	Meaning
m	Input byte message or block
d	Non-negative block integer
F_i	Fibonacci value with index i
I	Descending Zeckendorf indices
$t = I $	Number of selected Zeckendorf terms
G	Reduced index-gap sequence
$\mathcal{A}$	Ordered alphabet of distinct gaps
S	Alphabet-identifier sequence
w	Identifier width
$x y$	Concatenation
$ x $	Length in the stated unit

Table 3

Abbreviations used in the method and experiments.

Abbreviation	Expansion
FISA	Fibonacci Index-gap Shared Alphabet
FIPB	Fibonacci Independent Per-Block baseline
RLE	Run-length encoding
LEB128	Little-Endian Base-128 integer encoding

3.3.3. Reduced term indices

Taking adjacent differences gives

$$G = [22 - 15, 15 - 4, 4] = [7, 11, 4]. \quad (4)$$

The optimistic bit-length sum is $3 + 4 + 3 = 10$ bits. Thus the example follows $15 > 12 > 10$ at the representation level. The complete codec subsequently counts the alphabet, widths, lengths, padding, and framing omitted here. Algorithm 2 formalizes the decomposition and adjacent-gap reduction.

Input: Non-negative integer d

Output: Reduced gaps G

```

 $I \leftarrow []$ ;  $V \leftarrow \text{FibonacciTable}(d)$ ;
while  $d > 0$  do
    choose largest  $i$  such that  $V[i] \leq d$ ;
    append  $i$  to  $I$ ;  $d \leftarrow d - V[i]$ ;
end
for  $j \leftarrow 0$  to  $|I| - 2$  do
     $G[j] \leftarrow I[j] - I[j + 1]$ ;
end
if  $|I| > 0$  then
     $G[|I| - 1] \leftarrow I[|I| - 1]$ ;
end
return  $G$ ;

```

Algorithm 2: DecomposeAndReduce(d)

3.4. Alphabet extraction and binary transformation

For

$$G = [150, 200, 150, 300, 200, 250, 300, 150, 200],$$

the frequencies of 150, 200, 300, and 250 are 3, 3, 2, and 1. Sorting by decreasing frequency and then first occurrence gives

$$\mathcal{A} = [150, 200, 300, 250], \quad S = [0, 1, 0, 2, 1, 3, 2, 0, 1].$$

Since $|\mathcal{A}| = 4$, identifiers use $w = 2$ bits. The packed stream is

$$00||01||00||10||01||11||10||00||01.$$

Fixed width makes symbol boundaries unambiguous. The alphabet and its length are included in the payload. Algorithm 3 gives the frequency-ordered alphabet construction used by the reference implementation.

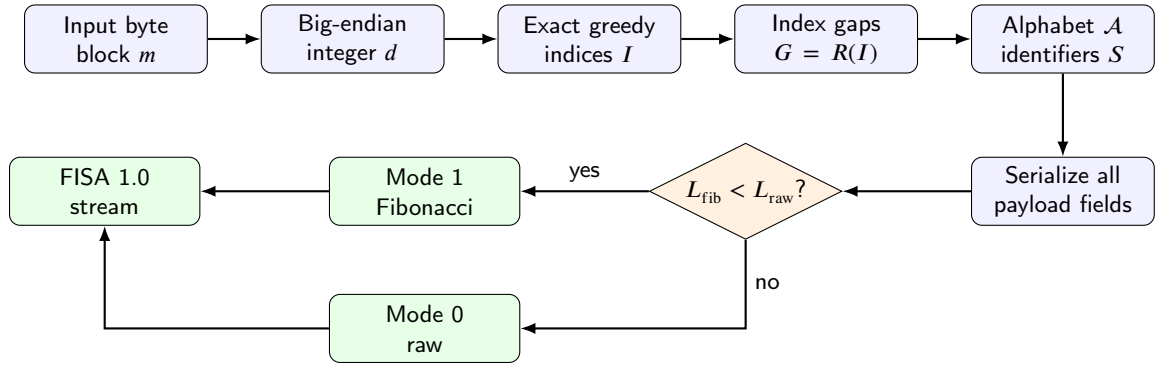


Figure 1: Blockwise encoding pipeline. All information needed by the decoder is serialized before the mode decision.

Input: Gap sequence G

Output: $\mathcal{A}$, identifiers S , width w

count values and record first positions;

$\mathcal{A} \leftarrow$ values sorted by decreasing count, then first position;

replace each $G[j]$ by its index $S[j]$ in $\mathcal{A}$;

$w \leftarrow \lceil \log_2 |\mathcal{A}| \rceil$ if $\mathcal{A} \neq \emptyset$, else 0;

return $(\mathcal{A}, S, w)$;

Algorithm 3: ExtractAlphabet(G)

4. Proposed Fibonacci index-gap approach

4.1. Encoding pipeline

Figure 1 presents the complete pipeline. The transform first produces an exact candidate payload. The selector then compares complete serialized lengths and stores either the Fibonacci payload or the raw block.

4.2. Exact decomposition

Let $(F_n)_{n \geq 0}$ be the Fibonacci sequence,

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2}. \quad (5)$$

We use the conventional positive terms $F_2 = 1, F_3 = 2, \dots$. For a non-negative block integer d , the greedy decomposition returns a descending sequence

$$I(d) = (i_0, i_1, \dots, i_{t-1}), \quad i_j - i_{j+1} \geq 2, \quad (6)$$

such that

$$d = \sum_{j=0}^{t-1} F_{i_j}. \quad (7)$$

For $d = 0$, the index sequence is empty. The implementation uses exact integer arithmetic and builds Fibonacci values iteratively until the block value is covered.

4.3. Index-gap transform

For a non-empty index sequence I , define

$$R(I) = G = (g_0, \dots, g_{t-1}), \quad (8)$$

where

$$g_j = i_j - i_{j+1} \quad (0 \leq j < t-1), \quad g_{t-1} = i_{t-1}. \quad (9)$$

The inverse is a suffix sum:

$$i_{t-1} = g_{t-1}, \quad i_j = g_j + i_{j+1}. \quad (10)$$

The gap values are frequently much smaller than the original indices. The representation builds an alphabet $\mathcal{A}$ from the distinct values in G , ordered by decreasing frequency with first occurrence as the tie-breaker. Each gap is replaced by its zero-based identifier in $\mathcal{A}$.

4.4. Independent per-block payload (FIPB baseline)

The independent per-block baseline, denoted FIPB, serializes a complete local Fibonacci payload for each block. It contains:

1. the alphabet size as an unsigned LEB128 integer;
2. the number of symbols as an unsigned LEB128 integer;
3. one byte for the fixed symbol width;
4. every alphabet value as an unsigned LEB128 integer;
5. the packed-stream length as an unsigned LEB128 integer; and
6. fixed-width identifiers packed most-significant bit first.

Padding bits in the final byte are zero and are included in the measured size. This explicit baseline prevents the evaluation from treating the local alphabet or delimiters as free information. FISA 1.0, defined next, instead amortizes one alphabet over the complete stream and does not repeat it inside each transformed block.

4.5. Shared-alphabet stream

Because repeating an alphabet in every block creates substantial overhead, FISA 1.0 constructs one frequency-ordered gap alphabet for all blocks in a stream. Each transformed block then stores only its symbol count, packed length, and identifiers. Local mode selection retains a block only when this reduced payload is shorter than the raw

block. A second stream-level decision compares the complete shared-alphabet stream with a raw stream, including the global alphabet and all framing:

$$L_{\text{FISA 1.0}}(x) = \min\{L_{\text{shared}}(x), L_{\text{raw-stream}}(x)\}. \quad (11)$$

Both branches are decoded and verified byte-for-byte in the experiments.

Proposition 1 (Exact amortized gain condition). *Let $N = \sum_i n_i$ be the input length, B the number of blocks, β the nominal block size, $a = |A|$, $w = \lceil \log_2 a \rceil$, and $u(z)$ the byte length of unsigned LEB128. If block i contains k_i gap symbols, define $q_i = \lceil k_i w / 8 \rceil$, $p_i = u(k_i) + u(q_i) + q_i$, and $s_i = \min\{n_i, p_i\}$. The complete shared representation is shorter than the raw stream if and only if*

$$u(\beta) + u(B) + u(a) + 1 + \sum_{g \in A} u(g) + \sum_{i=1}^B [1 + u(n_i) + u(s_i) + s_i] < N, \quad (12)$$

Proof. The terms in Eq. (12) are respectively the global block-size, block-count, alphabet-size and width fields, serialized alphabet values, and complete block records. The magic, format identifier, stream mode, and original-length field occur in both candidates and cancel. The remaining inequality is exactly $L_{\text{shared}} < L_{\text{raw-stream}}$. $\square$

Under equal amortization of the global metadata $C_A = u(\beta) + u(B) + u(a) + 1 + \sum_{g \in A} u(g)$, a computable sufficient threshold is

$$k^* = \max \{k : 1 + u(n) + u(p(k)) + p(k) + \lceil C_A / B \rceil < n\}, \quad (13)$$

with $p(k) = u(k) + u(\lceil kw / 8 \rceil) + \lceil kw / 8 \rceil$. Thus a term-count criterion is meaningful only when identifier width and amortized alphabet cost are also specified.

4.6. FISA 1.0 stream and fallback

The stream begins with the four-byte magic value FISA, a one-byte format identifier with value 1, a stream-mode byte, and the original byte length. Raw-stream mode stores the original bytes directly. Shared mode then stores the nominal block size, block count, global gap alphabet, and identifier width. Each block stores a mode byte, original block length, payload length, and payload. Mode 0 stores raw bytes; mode 1 stores shared-alphabet identifiers.

For comparison, the independent FIPB baseline chooses the mode of a block x according to

$$\text{mode}(x) = \begin{cases} \text{FIB}, & L_{\text{fib}}(x) < L_{\text{raw}}(x), \\ \text{RAW}, & \text{otherwise}, \end{cases} \quad (14)$$

where both lengths are known exactly after serialization. The strict inequality avoids choosing the more complex representation on a tie. Within a FISA shared stream, the analogous

Input: Byte block x

Output: Mode and payload

$d \leftarrow \text{IntegerFromBytes}(x)$;

$I \leftarrow \text{GreedyFibonacciIndices}(d)$;

$G \leftarrow R(I)$;

$(A, S) \leftarrow \text{Alphabetize}(G)$;

$p \leftarrow \text{Serialize}(A, S)$;

if $|p| < |x|$ **then**

return (FIB, p);

else

return (RAW, x);

end

Algorithm 4: Hybrid FIPB encoding of one independent block

local decision compares the shared-identifier body p_i with the raw block, without charging a local alphabet; the global alphabet and complete framing are then handled by Proposition 1.

Proposition 2 (Independent per-block gain condition). *For a block x in the FIPB baseline, Fibonacci mode is selected if and only if the independently serialized payload, including its local alphabet, lengths, symbol width, packed identifiers, and padding, occupies fewer bytes than x . The selected FIPB block payload never exceeds the corresponding raw payload. This proposition concerns the independent per-block selector in Eq. (14); the complete FISA shared-stream condition is Proposition 1.*

Proof. All Fibonacci fields are materialized before the comparison in Eq. (14). The encoder returns that payload only under the strict inequality $L_{\text{fib}}(x) < L_{\text{raw}}(x)$; otherwise it returns x . The selected payload length is $\min\{L_{\text{fib}}(x), L_{\text{raw}}(x)\}$. $\square$

Algorithm 4 implements this independent FIPB block selector.

4.7. Decoding processes

The FIPB decoder reverses the independent per-block mode. A raw block is copied directly. For a Fibonacci block, it parses the local alphabet and packed identifiers, reconstructs G , lifts suffix sums to recover I , and evaluates Eq. (7). The declared block length restores leading zero bytes. The FISA decoder performs the same inverse operations but parses the stream-level alphabet once and uses it for every shared-mode block. Algorithm 5 specifies the inverse FIPB block operation; FISA uses the same suffix-sum and Fibonacci reconstruction with the shared alphabet supplied by the stream header.

5. Correctness and analytical properties

This section checks the properties required by the format: bijectivity of the gap map, lossless reconstruction, the entropy consequences of a bijection, and computational complexity.

Input: Mode, payload, declared byte length ℓ
Output: Original block
if mode is RAW **then**
 | **return** payload;
end
 parse $(\mathcal{A}, |S|, w, S)$ from payload;
 $G[j] \leftarrow \mathcal{A}[S[j]]$ for every identifier;
 $I[|G| - 1] \leftarrow G[|G| - 1]$;
for $j \leftarrow |G| - 2, |G| - 3, \dots, 0$ **do**
 | $I[j] \leftarrow G[j] + I[j + 1]$;
end
 $d \leftarrow \sum_j F_{I[j]}$;
return d as exactly ℓ big-endian bytes;
Algorithm 5: DecodeFIPBlock(mode, payload, ℓ)

Proposition 3 (Bijectivity of index-gap reduction). *For every finite descending integer sequence I , the map R defined in Eq. (9) is bijective.*

Proof. Equation (10) reconstructs the final index directly and every preceding index recursively. Hence each gap vector has at most one preimage. Substituting Eq. (10) into Eq. (9) recovers every component of G , establishing existence and uniqueness. $\square$

Proposition 4 (Lossless block reconstruction). *Given a valid payload and the original block length, the decoder reconstructs the unique input block.*

Proof. The serialized alphabet maps each valid symbol identifier to one gap. The inverse transform reconstructs the unique index sequence. Summation of the exact Fibonacci terms in Eq. (7) reconstructs d . Conversion to the declared byte length restores leading zero bytes. The decoder rejects invalid identifiers, inconsistent lengths, truncated payloads, values that do not fit the declared block length, and trailing bytes. $\square$

5.1. Entropy interpretation

Let X be a discrete random variable and R a bijection. Then $H(R(X)) = H(X)$ because R only relabels outcomes and preserves their probabilities. For a block process (X_t) , componentwise invertibility also preserves every joint entropy $H(R(X_1), \dots, R(X_T)) = H(X_1, \dots, X_T)$ and preserves the entropy rate when the limit exists. It need not preserve the redundancy of a restricted coding model: symbol marginals, apparent empirical order, and factorization costs can change after transformation. Compression can improve when the transformed sequence is better matched by the chosen finite-context or alphabet code, without any reduction in source entropy. The empirical zero-order byte entropy reported here is descriptive rather than a universal finite-file lower bound.

5.2. Complexity

Let ℓ_d be the bit length of the block integer, t the number of selected Fibonacci terms, and k the number of distinct gaps. Exact fast doubling computes a Fibonacci term in $O(M(\ell_d) \log n)$ bit operations, where $M(\ell_d)$ is

the multiplication cost. The reference greedy method builds the required Fibonacci values once and performs at most $O(\ell_d)$ term selections because the largest index is $\Theta(\ell_d)$ and selected indices are non-consecutive. Gap construction is $O(t)$. Alphabet counting is expected $O(t)$, followed by $O(k \log k)$ ordering. The representation requires $O(\ell_d + t)$ bits, excluding interpreter overhead. In the measured implementation, decomposition dominates encoding time; decoding is cheaper when the raw-stream branch is selected.

6. Experimental protocol

The experiments below test the representation, its cost model, and the alternative explanations introduced in Section 2.

6.1. Research questions

The experiments address five questions:

- RQ1** For which integers does the gap representation reduce the optimistic bit count?
- RQ2** Which components determine the complete serialized size?
- RQ3** How does the hybrid representation compare with raw storage, universal integer codes, and established lossless codecs?
- RQ4** Is the measured effect specific to Fibonacci, or explained more simply by positional or zero-block structure?
- RQ5** How sensitive is the complete shared stream to block size?

6.2. Data

The synthetic corpus contains seven deterministic 64 KiB files generated with seed 20260608: zeros, ones, alternating bytes, a low-alphabet sequence, periodic text, a byte ramp, and pseudorandom bytes. Canterbury contains all 11 files in the standard archive. Silesia contains 12 heterogeneous files. Python sensitivity experiments use fixed 64 KiB prefixes; these results are labelled as ablations and are not used for full-corpus claims. The compiled FISA implementation evaluates every complete Silesia file. The 100,000,000-byte enwik8 file is evaluated by both FISA and the general-purpose baselines. Five deterministic monotone sequences of 100,000 integers cover dense, geometric, posting-like, and quadratic growth regimes.

The archive SHA-256 digests are recorded in the experiment manifest and are verified by the reproducibility scripts before corpus extraction.

6.3. Configurations and metrics

The independent per-block baseline is evaluated with block sizes of 16, 64, 256, and 512 bytes. The compiled shared-stream implementation evaluates the complete Canterbury and Silesia corpora at 16, 32, 64, 128, and 256 bytes. The 256-byte setting remains the common reference for

comparisons that require one fixed block size. The baselines are raw storage, byte RLE, zlib level 9, bzip2 level 9, LZMA preset 9 [17], Brotli quality 11 [18], Zstandard level 19, and PPMd order 6. Universal-code measurements include Elias gamma and delta [11], Fibonacci universal coding, unsigned LEB128, and an oracle Rice parameter [19, 20]. These universal-code values are payload-only and remain optimistic. Additional complete comparisons use canonical Huffman, signalled Rice, a stream-level shared alphabet, and Elias–Fano on monotone sequences.

The primary metric is

$$\rho = \frac{\text{serialized bytes}}{\text{input bytes}}. \quad (15)$$

We also record bits per byte, execution time, peak Python allocation, and empirical zero- and first-order conditional entropy. Every FISA output is decoded and compared byte-for-byte with the input. The implementation is tested over all integers from 1 to 9999, arbitrary byte strings, leading zeros, multiple block sizes, and malformed streams.

6.4. Paired comparison

Per-file comparison uses the complete shared-alphabet stream and the smallest modern baseline on the same file. Canterbury contributes 11 pairs and Silesia 12. We report win counts and median size factors. This finite-corpus summary does not treat files as a random sample from a wider population. Fixed-prefix measurements are used only for exploratory ablations.

6.5. AI-assisted tooling and verification

OpenAI Codex assisted with source-code editing, experiment orchestration, language editing, and manuscript-format checks. It did not supply experimental data or replace the stated algorithms. The authors inspected the changes, reran the deterministic workflows, verified every FISA stream by decoding it, and checked the reported tables against the machine-readable result files.

7. Results

7.1. Integer-range viability

Figure 2 compares the direct binary length with the sum of individual gap bit lengths for every integer from 1 through $d = 10^7$. This comparison excludes alphabets and framing and provides an upper bound on the attainable benefit. It characterizes only this finite range, whose largest values require about 24 bits; the much larger integers induced by blocks of up to 256 bytes are assessed through the corpus experiments rather than inferred from this enumeration.

The favorable fraction increases with the tested range and the optimistic mean crosses zero before 10^7 . This trend concerns only gap magnitudes; Eq. (12) remains the criterion for complete compression.

7.2. Is the effect specific to Fibonacci?

Table 4 applies the same optimistic gap-bit metric to three index systems over $1 \leq d \leq 10^6$. The canonical

Fibonacci result is not optimal under this metric. A greedy Lucas decomposition has a larger favorable fraction, while set-bit positions in the ordinary binary representation are shortest for most integers. The latter omits complete delimiters, as do the Fibonacci and Lucas rows, and serves as a structural control rather than a deployable codec.

This comparison changes the interpretation. Fibonacci provides a canonical non-consecutive decomposition, but the gain mechanism is position-gap coding under an amortized model. The Fibonacci basis is one exactly reversible instance, not an empirically optimal basis.

7.3. Empirical calibration of the term threshold

Table 5 calibrates Eq. (13) on complete files. Because each file has its own alphabet and amortized metadata, k^* is a stream-level quantity rather than one corpus-wide constant. At 256 bytes it is 390–392 terms on Canterbury and 326 terms on every Silesia file. Only 3.70% and 1.94% of complete blocks satisfy the sufficient bound. These fractions closely track the 3.75% and 1.94% of all blocks whose local shared payload is actually selected.

The close agreement does not turn the sufficient condition into a necessary one. It shows that, under the measured alphabet widths and low amortized metadata, term count is the dominant local selector. It also quantifies why the favorable regime occupies only a small minority of standard-corpus blocks.

7.4. Ablation of representation costs

Table 6 separates three quantities on Canterbury and fixed Silesia prefixes: the bit lengths of the Fibonacci indices, the bit lengths of the reduced gaps, and the complete serialized payload. Gap reduction consistently improves on direct index storage. On Canterbury with 256-byte blocks, the gap ratio is 0.9439 while the complete payload ratio is 1.6279. The 0.6840 difference identifies the cost of alphabet values, identifiers, lengths, width fields, and byte padding.

7.5. Entropy coding and shared alphabets

Figure 3 compares coding methods on the same 256-byte Fibonacci gap streams. Independent canonical Huffman and Rice codes reduce the cost relative to the original fixed-width alphabet payload but remain above raw size because each block still signals a model or parameter. Sharing the alphabet changes the weighted ratio from 1.5849 to 0.9713 on Canterbury and from 1.5955 to 0.9608 on fixed 64 KiB Silesia prefixes. The payload-only entropy lower bounds are 0.9415 and 0.9483, respectively. The complete-corpus result is reported separately in Table 8.

On complete files, the shared stream compresses ptt5 to 0.8424, mozilla to 0.9938, and mr to 0.7732. The apparent gain on the 64 KiB ooffice prefix does not persist on the complete file. The mozilla ratio likewise changes from 0.7672 on its fixed prefix to 0.9938 on the complete file, an absolute increase of 0.2266. This change indicates that the prefix has a substantially higher concentration of favorable structure than the remaining file. Other files select the stream-level raw branch. These cases justify restricting

Limits of Fibonacci Index-Gap Coding

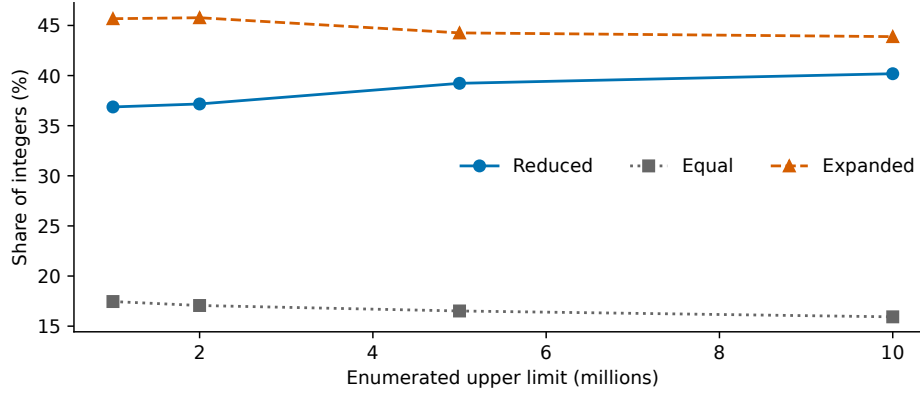


Figure 2: Integer-range scaling. Fractions of enumerated integers whose delimiter-free Fibonacci gap length is shorter than, equal to, or longer than the direct binary length.

Table 4

Optimistic index-gap comparison over $1 \leq d \leq 10^6$.

Basis	Gain (%)	Equal (%)	Expansion (%)	Mean bits	Mean terms	Unique best (%)
Fibonacci (Zeckendorf)	36.87	17.46	45.67	19.0818	7.8945	5.36
Lucas (greedy)	48.35	18.07	33.59	18.4216	8.1328	9.62
Binary set-bit positions	98.39	1.61	0.00	15.4673	9.8850	73.67

Note: "Unique best" is the fraction for which one method alone has the shortest measured gap sequence.

prefix results to sensitivity analysis and basing corpus claims on complete inputs.

7.6. Universal-code comparison

Replacing fixed identifiers with a standard universal code does not remove the limitation. At a 256-byte block size on Canterbury, the optimistic payload-only ratios are 0.9439 for delimiter-free fixed binary gap values, 1.1612

for oracle Rice coding, 1.5007 for Elias gamma, 1.5152 for Fibonacci universal coding, 1.7612 for Elias delta, and 3.0960 for LEB128. The same ordering is observed on fixed Silesia prefixes. The fixed-binary figure is not a complete decodable stream because it omits delimiters; it serves only to isolate the intrinsic gap magnitudes.

Table 5

Empirical calibration of the sufficient term threshold.

Corpus	Files	k^* range	Complete blocks	Bound satisfied (%)	All blocks	Shared selected (%)
Canterbury	11	390–392	10,975	3.699	10,986	3.750
Silesia	12	326	827,882	1.937	827,890	1.938

Note: The calibration uses complete 256-byte blocks. The selected fraction includes final partial blocks.

Table 6

Exploratory mean ablation ratios.

Corpus	Block (bytes)	Indices	Gaps	Payload
Canterbury	16	2.4250	0.8892	1.9046
Canterbury	64	3.2478	0.9189	1.7424
Canterbury	256	4.1008	0.9439	1.6279
Canterbury	512	4.5120	0.9496	1.6377
Silesia sample	16	2.4771	0.9024	1.9354
Silesia sample	64	3.2404	0.9151	1.7347
Silesia sample	256	4.0288	0.9252	1.5955
Silesia sample	512	4.5015	0.9452	1.6370

Note: The Silesia measurements use fixed 64 KiB prefixes. "Payload" includes all fields inside the Fibonacci block payload but excludes the outer block record.

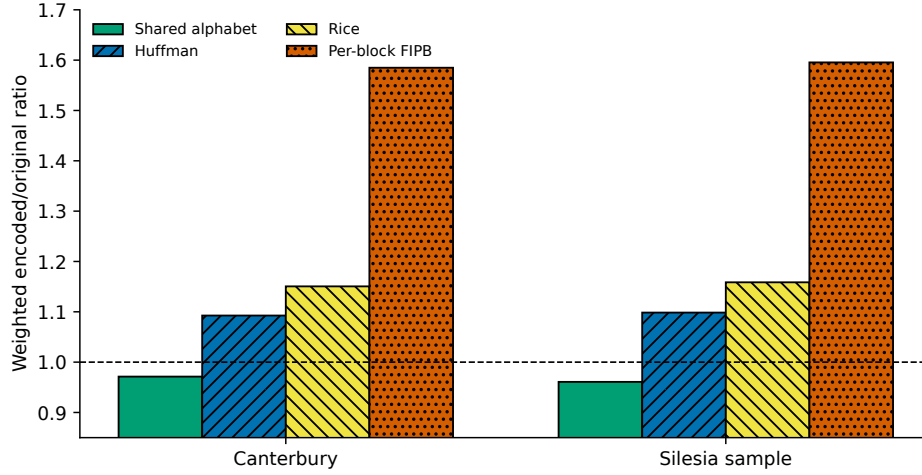


Figure 3: Weighted size comparison on identical 256-byte Fibonacci gap streams. The Silesia values use fixed 64 KiB prefixes; full-corpus results appear in Table 8.

Table 7

Complete shared-stream sensitivity to block size.

Block	Canterbury			Silesia		
	Ratio	Files < 1	Shared (%)	Ratio	Files < 1	Shared (%)
16	0.9274	1	12.956	0.9978	1	3.084
32	0.9215	1	11.353	0.9930	1	2.745
64	0.9354	1	9.136	0.9905	1	2.497
128	0.9593	1	6.100	0.9900	1	2.227
256	0.9713	1	3.750	0.9878	2	1.938

Note: Ratios include all stream and block fields. "Shared" is the fraction of blocks that select the gap representation.

7.7. Corpus-level compression

Table 7 reports the full-corpus block-size sensitivity. Canterbury is smallest at 32 bytes, whereas Silesia is smallest at 256 bytes. The difference follows from two competing costs. Short blocks make sparse local patterns easier to isolate but create more block records; long blocks amortize those records but usually contain more Zeckendorf terms. No tested size makes the method broadly applicable. Only one Canterbury file compresses at every size, and Silesia has one compressed file through 128 bytes and two at 256 bytes.

The remaining cross-method tables use 256 bytes as a common reference rather than selecting a different size for each corpus. At that setting, FISA reaches 0.9713 on Canterbury and 0.9878 on Silesia. The complete ooffice file selects the raw-stream branch, showing why prefix results cannot be extrapolated to a full file.

Modern codecs win every paired file comparison. These comparisons define the scope of the shared representation: it is a conditional transform rather than a general-purpose codec.

The modern-codec oracle wins all 11 Canterbury comparisons and all 12 Silesia comparisons. The median FISA-to-oracle ratio factors are 3.92 and 4.23. These win counts

establish the direction of the measured result on the evaluated files; the small corpus counts do not support broad distributional inference.

For context, full-corpus weighted ratios on Canterbury are 0.1746 for Brotli, 0.1751 for LZMA, 0.1819 for PPMd, 0.1837 for Zstandard, 0.1931 for bzip2, and 0.2589 for zlib. On full Silesia, the corresponding ratios are 0.2375, 0.2302, 0.2367, 0.2497, 0.2572, and 0.3192.

7.8. Sparse-structure controls

Zero dominance alone is not a sufficient application criterion. In a controlled 1 MiB corpus with independently positioned nonzero bytes, FISA has ratio 0.8650 at 1% density and selects raw storage from 5% density upward. At 1% density, the measured general-purpose ratios range from 0.0217 for PPMd to 0.0327 for zlib.

Figure 4 compares FISA with two complete, exactly decodable controls after the same block-size sweep. The bitmap method stores one presence bit per block and copies nonzero blocks. The trim method stores the leading-zero count and the middle span of every block. Both controls attain a smaller corpus ratio than FISA.

The matched sweeps strengthen the structural explanation. On Canterbury, the best bitmap and trim streams save about 2.8 percentage points beyond the best FISA stream.

Table 8

Complete-corpus comparison at the common 256-byte FISA setting.

Corpus	Files	Weighted shared	Best weighted modern
Canterbury	11	0.9713	0.1746
Silesia complete	12	0.9878	0.2302
enwik8	1	≈ 1.000	0.2483

Note: The modern column selects the smallest tested codec for each file.

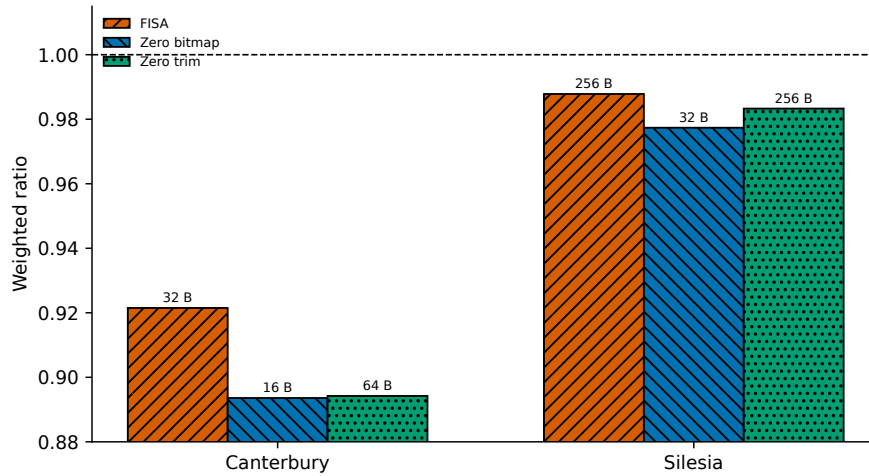


Figure 4: Structural controls at their best tested block sizes. A ratio below one denotes compression; lower values indicate smaller complete streams. Labels above the bars give the selected block size in bytes.

On Silesia, the best bitmap saves about 1.0 percentage point. Block-level zero structure accounts for the favorable cases with less machinery than Fibonacci decomposition.

7.9. Monotone integer sequences and enwik8

Elias–Fano is evaluated only on its intended domain: strictly increasing integer sequences. On five deterministic 100,000-value datasets it requires 2.00, 3.00, 4.25, 3.52, and 18.53 bits per integer for dense, geometric ($p = 0.5$), geometric ($p = 0.2$), posting-like, and quadratic sequences. Complete gap Rice coding ranges from 1.00 to 18.03 bits per integer, while complete gap Huffman is best on the posting-like sequence at 1.82 bits per integer. These results show that no single integer representation dominates: the source distribution and monotonicity determine the appropriate code.

On the 100,000,000-byte enwik8 corpus, complete ratios are 0.2483 for PPMd, 0.2487 for LZMA, 0.2694 for Zstandard, 0.2705 for Brotli, 0.2901 for bzip2, 0.3648 for zlib, and approximately 1.000 for compiled FISA; the latter selects the raw-stream branch and differs from unity only by stream framing. This benchmark confirms the fallback behaviour on an unfavorable large input.

7.10. Entropy and context

Mean empirical zero-order entropy is 4.4051 bits/byte on Canterbury and 5.0498 bits/byte on fixed Silesia prefixes. The corresponding first-order conditional values are 3.0015 and 3.4052 bits/byte. The reductions of 1.4036 and 1.6446 bits/byte indicate substantial sequential dependence. The

current block integer transform does not explicitly model this dependence, whereas dictionary and context-based codecs can exploit it.

8. Discussion

8.1. Why compression remained rare

A favorable block needs a sparse Zeckendorf decomposition and repeated gap values. Even then, the saved bits must cover its record and a share of the file-level alphabet. Most text and binary blocks already fail at sparsity. Zero-dominated blocks sometimes pass, but the bitmap and trim controls encode the same structure with less machinery.

The mozilla file gives a concrete warning. Its first 64 KiB produced a ratio of 0.7672, which looked useful. The complete file reached 0.9938. The prefix happened to contain much more favorable structure than the rest of the input. A result based on that prefix alone would have overstated the method’s practical range.

Changing the block size did not rescue the representation. Short blocks isolate local zero regions but create more records. Long blocks reduce that record cost and select more Fibonacci terms. Canterbury favored 32 bytes; Silesia favored 256. At either optimum, almost every file still used the raw branch.

8.2. What byte-level accounting revealed

The gap statistic and the serialized stream answer different questions. Several delimiter-free gap lists are shorter

than raw binary, but they do not identify symbol boundaries and they carry no alphabet. Equations (12) and (13) put those omitted bytes back into the comparison. They show where a positional transform stops saving space once its model and records are present.

8.3. Comparison with established coding methods

FISA converts each block to one large integer and loses much of the repetition and context that dictionary codecs exploit. The measured gap to those codecs is consistent with that design choice. Fibonacci universal coding solves a different problem because it self-delimits positive integers; FISA instead stores gaps under a shared alphabet. Binary-position gaps are the closer structural comparison. Their result makes clear that the favorable integer count is not a Fibonacci effect.

8.4. Limitations

The Python implementation remains a clarity-oriented reference. A compiled Java implementation with cached Fibonacci tables evaluates all 211.9 MB of Silesia and the 100 MB enwik8 file. Runtime measurements are retained in the reproducibility package, but no cross-codec speed claim is made because the evaluated baselines use different runtimes and bindings.

The format uses one stream-level alphabet and fixed-width identifiers. Hierarchical alphabets, canonical entropy codes over the shared identifier stream, or context models may further reduce overhead, but they would require separate specification and measurement. The integer sweep covers the range up to 10^7 ; larger block integers are sampled through the corpus experiments rather than exhaustively enumerated.

8.5. Implications

The evidence does not support further optimization of FISA as a general-purpose codec. A more promising continuation is an adaptive sparse block representation that selects directly among zero bitmaps, boundary trimming, binary positions, and raw storage. The cost equations and decode-after-encode checks remain useful for evaluating such a system.

9. Conclusion

We asked whether Fibonacci index gaps remain smaller after a decoder's full input is serialized. Usually, they do not. FISA is reversible, and its best weighted ratios are 0.9215 on Canterbury and 0.9878 on Silesia, but only one and two files compress. Every established codec wins its paired comparisons.

The controls explain the few positive cases. Binary set-bit positions improve the integer-level statistic without using Fibonacci. On complete files, zero-block bitmaps and boundary trimming are smaller than FISA. The main result is a boundary, not a new general-purpose codec: sparse index gaps can look compact while their alphabet and records erase the saving. Future work should start from the observed

zero structure and compare its serialized cost directly, before assigning the gain to a number system.

Acknowledgements

This work was supported by the authors' institutions. No external financial support was received.

CRedit authorship contribution statement

Stéphane Gaël R. EKOdeck: Conceptualization, methodology, software, formal analysis, investigation, data curation, visualization, writing – original draft, writing – review and editing, supervision, project administration. **Hervé Talé KALACHI:** Methodology, formal analysis, validation, writing – review and editing. **Serge Alain EBÉLÉ:** Conceptualization, methodology, formal analysis, validation, writing – review and editing. **Norbert DJONG WANG:** Investigation, resources, data curation, validation, writing – review and editing. **Chantal Marguerite MVEH-ABIA:** Methodology, validation, supervision, project administration, writing – review and editing.

Declarations

Ethics approval. This study does not involve human participants, animals, personal data, or clinical interventions. It uses publicly available benchmark corpora, synthetic data, and offline computational experiments.

Competing interests. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding. This research received no external funding.

Data availability. The Canterbury corpus, Silesia corpus, and enwik8 benchmark are publicly available from their original distributors under their applicable terms. The fixed samples, synthetic-data generators, aggregate results, and per-file evaluation artefacts accompany the reproducibility package.

Code availability. The reproducibility package contains the reference implementation, tests, experiment configurations, evaluation scripts, machine-readable result tables, figure-generation code, and exact random seeds. It is publicly available at <https://github.com/EkodeckStephane/fibonacci-index-gap-coding>. An archival DOI will be added when the repository is deposited.

A. Container fields

Table 9 lists every serialized field in FISA 1.0.

A.1. Structural grammar

The following ABNF-like grammar uses `uvarint` for the shortest unsigned LEB128 form and `byte{n}` for exactly n bytes.

Table 9
FISA 1.0 shared-stream fields.

Scope	Field	Encoding
Stream	Magic	ASCII FISA
Stream	Format identifier	One byte, value 1
Stream	Stream mode	One byte: 0 raw, 1 shared
Stream	Original byte length	Variable-length integer
Raw stream	Payload	Exactly the original byte length
Shared stream	Nominal block size	Variable-length integer
Shared stream	Number of blocks	Variable-length integer
Shared stream	Alphabet size	Variable-length integer
Shared stream	Symbol width	One byte
Shared stream	Alphabet values	Variable-length integers
Block	Mode	One byte: 0 raw, 1 shared identifiers
Block	Original block length	Variable-length integer
Block	Payload length	Variable-length integer
Block	Payload	Declared number of bytes
Shared payload	Symbol count	Variable-length integer
Shared payload	Packed length	Variable-length integer
Shared payload	Symbol identifiers	Fixed-width, MSB first

Note: All variable-length integers use unsigned LEB128.

```

stream      = "FISA" version stream-mode
              original-length body
body        = raw-stream / shared-stream
version     = %x01
stream-mode = %x00 / %x01
raw-stream  = byte{original-length}
shared-stream = block-size block-count
              alphabet-size width alphabet block-list
alphabet    = uvarint{alphabet-size}
block-list  = block{block-count}
block       = block-mode block-length payload-length
              block-payload
block-mode  = %x00 / %x01
block-payload = raw-payload / shared-body
raw-payload = byte{payload-length}
shared-body = symbol-count packed-length
              byte{packed-length}

```

The selected branch is determined by `stream-mode`. In shared mode, the `block-mode` field determines the conditional alternative: mode 1 requires `block-payload = shared-body`, whereas mode 0 requires `block-payload = raw-payload` and `payload-length = block-length`. For mode 1, the bytes consumed by `shared-body` must equal `payload-length`.

A.2. Degenerate cases and mandatory errors

An empty input has original length zero, zero blocks, and is represented by the raw stream branch. A zero-valued nonempty block has an empty gap sequence; its declared block length preserves all leading zero bytes. The decoder must reject an unknown magic, format identifier, stream mode, or block mode; truncated or unreasonably long length fields; zero block size in shared mode; symbol identifiers outside the alphabet; packed lengths inconsistent with symbol count and width; reconstructed integers exceeding their block length; incorrect final length; and trailing bytes after the declared stream.

B. Reproducibility checklist

- Python 3.13.5 and a Java 8-compatible compiled implementation; package versions are pinned in the experiment dependency manifest.
- Synthetic seed: 20260608.
- Exact decode-after-encode verification for every measured FISA stream.
- Final automated test result: 10,119 passed.
- Bootstrap iterations: 10,000.
- All output sizes include stream and block framing for complete-codec results. Payload-only universal-code comparisons are labeled explicitly.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used OpenAI Codex to assist with source-code editing, experiment orchestration, language editing, formatting, and consistency checks. After using this service, the authors reviewed and edited the outputs, reran the reported workflows, verified the results against the source data, and take full responsibility for the content of the publication.

References

- [1] E. Zeckendorf, Représentation des nombres naturels par une somme de nombres de Fibonacci ou de nombres de Lucas, *Bulletin de la Société Royale des Sciences de Liège* 41 (1972) 179–182.
- [2] A. Moffat, Huffman coding, *ACM Computing Surveys* 52 (2019) 1–35.
- [3] T. C. Bell, J. G. Cleary, I. H. Witten, *Text Compression*, Prentice Hall, Englewood Cliffs, NJ, 1990.
- [4] I. H. Witten, A. Moffat, T. C. Bell, *Managing Gigabytes: Compressing and Indexing Documents and Images*, Morgan Kaufmann, 2 edition, 1999.
- [5] M. Burrows, D. J. Wheeler, A Block-Sorting Lossless Data Compression Algorithm, Technical Report SRC Research Report 124, Digital Equipment Corporation Systems Research Center, 1994.
- [6] P. M. Fenwick, The Burrows–Wheeler transform for block sorting text compression: Principles and improvements, *The Computer Journal* 39 (1996) 731–740.
- [7] Y. Collet, M. Kucherawy, Zstandard compression and the ‘application/zstd’ media type, RFC 8878, 2021.
- [8] J. Ellert, J. Fischer, M. R. Pedersen, New advances in rightmost Lempel–Ziv, in: *String Processing and Information Retrieval*, Springer, 2023, pp. 188–202.
- [9] A. S. Fraenkel, S. T. Klein, Robust universal complete codes for transmission and compression, *Discrete Applied Mathematics* 64 (1996) 31–55.
- [10] N. R. Brisaboa, S. Ladra, Ó. Pedreira, J. R. Trillo, On the practical use of Fibonacci codes for indexing, *Journal of Intelligent Information Systems* 56 (2021) 313–341.
- [11] P. Elias, Universal codeword sets and representations of the integers, *IEEE Transactions on Information Theory* 21 (1975) 194–203.
- [12] D. Shapira, New compression schemes for natural number sequences, *Discrete Applied Mathematics* 324 (2023) 96–110.
- [13] P. Pooksommat, P. Udomkavanich, W. Kositwattanarek, Multidimensional Fibonacci coding, *Mathematics* 10 (2022) 1037.
- [14] T. Pelkonen, S. Franklin, P. Cavallaro, Q. Huang, J. Meza, J. Teller, K. Veeraraghavan, Gorilla: A fast, scalable, in-memory time series database, *Proceedings of the VLDB Endowment* 8 (2015) 1816–1827.
- [15] P. Liakos, K. Papakonstantinou, Y. Kotidis, Chimp: Efficient lossless floating point compression for time series databases, *Proceedings of the VLDB Endowment* 15 (2022) 3058–3070.
- [16] D. W. Blalock, S. Madden, J. V. Guttag, Sprintz: Time series compression for the IoT, *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 2 (2018) 93:1–93:23.
- [17] I. Pavlov, LZMA specification, LZMA SDK specification, 7-Zip, 2015. Draft specification released 14 June 2015.
- [18] J. Alakuijala, Z. Szabadka, Brotli compressed data format, RFC 7932, 2016.
- [19] S. W. Golomb, Run-length encodings, *IEEE Transactions on Information Theory* 12 (1966) 399–401.
- [20] R. F. Rice, J. R. Plaunt, Adaptive variable-length coding for efficient compression of spacecraft television data, *IEEE Transactions on Communication Technology* 19 (1971) 889–897.